

WOOC '93 資料

信号処理に向けたオブジェクトモデルの提案と応用

*安賀 広幸 今井 克暢† 下條 真司‡ 宮原 秀夫

大阪大学 基礎工学部 情報工学科

†大阪大学 基礎工学部 生物工学科

‡大阪大学大型計算機センター

Abstract

本稿では、音声、画像を含むマルチメディアデータの主要な部分である信号を構造をもったデータの集合を表現する関数と考え、それに対する柔軟な操作を定義する新しい枠組みを提案する。

現状の信号処理のプログラムは配列とループによって記述されており、信号処理の理論的な構成要素である、信号、変換、あるいはその伝達関数の組合せとは大きく異なったモデルとなっている。そこでまず、信号を関数として捉えることで、信号の意味構造に基づいた抽象的な扱いを可能とする。さらに、これによってある計算の結果を部分的に必要なときには、関数の構造をたどることによって、容易に元となる集合を特定し、再計算を部分的に行なうことが可能となる。また、信号に対する処理をモジュールの結合として表現することで信号処理に向けたデータフロー的な処理の記述が可能となる。特に、信号の変換に対応する伝達関数を表現することができる。さらに、信号上の領域を扱えるようにすることで、信号をある性質を持った領域に分割し、その性質に応じてメソッド選択を行なうことを可能とする。これによって、従来信号処理で、そのデータの持つ性質に合わせて、手法やパラメータを使い分けなければならないような試行錯誤的で複雑な処理を素直に表現することができる。

1 はじめに

計算機の非常に重要な応用分野として、アナログデータをサンプル・量子化することによって計算機に扱える形に変換し、計算処理を行うことが挙げられる。それに加えて、最近では、テキスト、音声、画像を含むマルチメディア情報全てを計算機上で統合的に処理する技術の研究・開発が盛んに行なわれている。

これらの処理の対象となるデータに共通する特徴として、扱わなければならないデータ量が非常に多い一方で、数値データ 1 つ 1 つのもつ意味は、従来のデータモデルの議論の中で扱われてきたプリミティブなデータよりも小さく、ある程度集合としてまとまった塊になってはじめて意味をもってくることが多いことがあげられる。例えば、音声信号のデータでは、個々の信号はある瞬間の電圧の値であり、1 つ 1 つを取り出しても、それ単体ではほとんど意味をなさない。以上のような特徴をもったデータを包括的にマルチメディアデータとよぶことにする。

このようなデータに対する処理は、そのデータの持つ性質に合わせて、手法やパラメータを使い分けなければならないが、現状では適切な手法やパラメータは経験的に求められており、そのため試行錯誤を繰り返すことになる。ところが、現状の信号処理のプログラムは配列とループによって記述されており、信号処理の理論的な構成要素である、信号、変換、あるいはその特性関数の組合せとは大きく異なったモデルとなっている。このため、試行錯誤にともなう手法やパラメータの変更が行ないにくく、またそのプログラムを使って得られた実験結果の管理が困難となっている。

*E-mail : aga@ics.osaka-u.ac.jp

このため、信号のような構造をもったデータの集合を取り扱うためのモデルが求められているわけであるが、従来のオブジェクト指向言語におけるクラスでこのような集合を表現することには無理がある。

しかし、信号に対する一般的な取り扱いを考える以上、普遍的な集合の扱いとその構造の表現法が必要であり、これらはオブジェクト指向言語あるいはデータベースのデータモデルによって提供されるべき機能であると思われる。

そこで本稿で提案するモデルでは、集合に対してメソッドを付加することを考える。すると、その集合の持つ構造的な性質は、付加されるメソッドの集合によって表現することが出来る。この時、後者関数や距離のような、異なる集合に対して付加されているメソッドではあるが、それぞれの集合に対して同じ意味を持つものが存在するので、これらのメソッドをまとめた総称関数を定義することを考える。そして信号を関数として捉え、それに対する柔軟な操作を定義する枠組みを提案する。

ここで提案している処理の記述は、ストリーム や CCL [9] を元にしたもので、信号データに対する処理をデータフロー的に記述する。これは、信号に対する変換の合成を、特性関数間の演算によって表現する方法に似ており、手法やパラメータを変更しながらおこなう試行錯誤に向いていると考えられる。また関数として信号を表現しており、集合の構造を保存することから、ある計算の結果を部分的に必要なときには、関数の構造をたどることによって、容易に元となる集合を特定し、再計算を部分的に行なうことが可能となる。

2 マルチメディアデータの処理の特徴

マルチメディアデータの処理と一口に言ってもその内容はいろいろであるが、ここでは、まず重要なマルチメディアデータである信号の性質について述べ、その後、これらのデータを信号処理の対象として取り扱うときの処理の特徴について考えていくことにする。

2.1 信号と信号処理

上で述べたように、マルチメディアデータには色々な種類があるが、その中でも重要なものの1つとして信号 (signal) がある。ここで述べる信号は、信号処理の分野における信号のことであり、アナログデータをサンプリングし量子化することによって得られるデジタルデータのことである。このようなデータに対する処理、すなわち信号処理は大きく3つの段階に分けて考えることができる (文献 [3])。

(1) 信号のデジタル化と符号化:

- 信号を連続した形から離散的な形 (デジタル化) すること
- その結果を“圧縮”して記憶量や通信路容量を節約すること

(2) 信号の強調と復元: 画像の劣化 (ぼけ, 雑音) を改善すること

(3) 信号の分割と記述:

- 画像を単純な“図面”に変換すること
- 画像あるいはその部分の特徴を測定すること
- 画像をその部分と特徴によって分類し記述すること

信号のデジタル化では、まずアナログデータをサンプリングによって、時間的あるいは空間的に量子化する。音声ならば一定の時間間隔に、画像ならピクセルになる。ただ、信号の強度はまだ量子化されておらずアナログのままなので *sampled analog* と呼ばれる。次に、信号の強度を量子化する。ここまできて初めてデジタルデータとしての信号となる。

このようにして得られた信号の特徴として、扱わなければならないデータ量が非常に多い一方で、数値データ1つ1つの持つ意味は、従来のデータモデルの議論の中で扱われてきたプリミティブなデータよりも小さく、ある程度まとまった塊になってはじめて意味をもってくることが多いことがあげられる。例えば、音声信号のデータでは、個々のデータはある瞬間の電圧の値であり、1つ1つを取り出しても、それ単体ではほとんど意味をなさない。

そこで、粒度の小さいデータを扱いやすくするため、信号の分割と記述という段階を通して、信号に含まれる個々のデータが集まって、どのような特徴を持っているかを抽出し、意味構造を組み立てる。このとき、信号の強調と復元という段階は、特徴抽出を行ないやすくするための前処理ということができる。すると、信号の数値データの系列が1つの意味に集約されるため、データの粒度を大きく、量は少なくなり、アプリケーションから使うことができるようになる。基本的にはこれが信号処理の考え方と言えるだろう。

先ほどの音声の例について言えば、22KHzでサンプリングしてデータの場合、人の声の1周期は100-300サンプルであり、これが10回とか連続することによってはじめて、“あ”という意味を持つことになる。

2.2 信号処理における問題点

ここまで述べてきた信号処理における問題点は、次の3つではないかと思われる。

- (1) データ構造の表現力の問題
- (2) 原信号の例外的な特徴への対処
- (3) アプリケーションからの要求

(1)は、信号処理の理論が前提としている離散的な信号の表現能力に対して、計算機上のデータ構造の表現能力が貧困であるために起こる問題と言える。良く起きる問題の例としては、信号処理のアルゴリズムが無限の信号系列を仮定しているのに対して、実際のデータ構造が有限のデータであるため、その境界の扱いとデータの補間が細かいバグを引き起こすと言ったものがある。この問題を、対処療法的なプログラミングによって解決すると、非常に保守性の悪いプログラムとなるため、信号処理のプログラムを記述・修正するのに適したデータモデルを導入することを考えなければならない。

(2)は、信号処理を含む処理の開発を行なう際における問題で、特徴抽出の本質的な部分に関わる問題である。

信号処理において特徴とは、人間にとって何か特定の意味を持つと考えられる部分の持つ性質であり、信号に対して厳密な定義がなされているわけではない。大抵の場合は、何らかの特徴を反映していると思われる値を信号から計算し、適当な閾値を使って場合分けを行ない、特徴と思われるものを見つけ出している。つまり、特徴量の計算式や閾値はきわめて経験的に知られているものである。そこで、実験データに各種の手法を適用して、その信号に特有な少し変わった特徴や、測定誤差や個体差などのエラーを含んでおり、一般的な手法をそのまま適用することができなくなってしまうような、例外的な状況が発生すると、適切な手法やパラメータを試行錯誤によって求める必要がある。そこで、信号処理のシステムの保守性が問題となる。

(3)は、アプリケーションから信号処理への2種類の要求に基づいている。

- アプリケーションから信号処理への要求・ヒント
- 巨大なデータの扱い

従来、信号処理は音声認識などが目的だったため、アプリケーションは信号処理プログラムが出力する意味的構造を受けとるだけであった。しかし、最近注目されているマルチメディアのアプリケーションなどでは、信号処理によって得られた意味的構造をもとにして、信号データそのものに情報を付加しようとしている。また音声認識などの分野でも、アプリケーションから認識へのヒントになるような情報を与えて、信号

処理の戦略を切替えるといったことも考えられる。そのため、信号処理によって抽出された意味的構造と原信号との間の対応関係を保存する必要がある。出てきている。

一方、技術的な進歩にともなって、扱うべきデータ量は増大する一方であり、数十から数百 MByte のデータを扱うのはごく一般的なこととなっている。このデータ量の多さへの対処は、大きく分けて次に 2 つの方向がある。

- 処理速度の向上
- 階層的な特徴抽出

処理速度の向上は、アルゴリズムの改良、プログラムの最適化、部分的な計算など計算機上における実装面での問題と言え、本稿の主題ではない。

これに対して階層的な特徴抽出とは、精度は荒いが計算が速い手法と、精度は細かいが計算が遅い手法を組み合わせ、重要と思われる部分の処理を集中して行なうような方法のことである。これは信号処理のパラメータを決める時のように、試行錯誤が多い場面で有効と考えられるが、“重要と思われる部分”を特徴抽出された意味的構造に基づいて判断しようとするものであり、やはり意味的構造から原信号への対応関係を保存する必要がある。

本稿では、従来のプログラミング言語よりも、信号処理の記述に適したデータモデルを導入することによって、信号処理の各アプリケーション毎にではなく、信号処理プログラム全体に対して、これらの問題点を解決する方法を考えていく。

3 信号とその処理のモデル化

本稿では、ここまで述べてきたような信号処理の問題点について、次のような方法によって解決することを考える。

- 信号を関数として表現する
- 信号に対する処理をモジュールの結合として表現する
- 信号上の領域を扱えるようにする

その後、信号を領域に分割し個々の領域について性質を記述し、その性質に合わせて処理を最適化することを考えることにする。

3.1 関数としての信号

信号処理の理論では信号を関数として扱っているの、計算機上でも関数として扱えることが望ましい。そこで、信号に関数のインターフェースを定義しカプセル化することで、配列を使って信号を表現したときの問題点を解消できると考える。

この時の問題点は、意味的構造のデータモデルをどう扱うかである。信号のデータを関数としてモデル化すれば、信号処理の理論に沿ったものになるが、特徴抽出して得られる意味的構造はグラフであり、信号とは大きく異なっている。

そこで、信号やグラフのようなデータの集まりを値と構造に分けて考えることにする。まず、これらのデータの集まりに含まれている各要素には識別子 (identifier) が存在するものとする。そして識別子から値への対応関係を関数として表現し、構造すなわち識別子間の関係は関数の定義域の持つ構造と考えることにする。

すると、信号の場合は、識別子は時刻 n のことであり、その構造は整数と同じものであると考えられる。また意味的構造を表現するグラフの場合は、識別子は各ノードの id に、その値は各ノードに付随する属性に対応し、識別子間の関係が辺によって表現されていると考えることが出来る。

こうすることにより、信号と意味的構造とを関数として統合的に扱うことが出来るようになる。ただし、構造化プログラミングなどでは、関数呼び出しのオーバーヘッドは極めて小さいといわれているが、信号処理の場合は信号データの量が多いため、各時刻における信号の値の参照が関数呼び出しに置き換わると、非常に性能が低下すると考えられるので、信号に対する処理の記述法を併せて検討しなければならない。

3.2 データフロー的な処理の記述

一般に、信号処理のシステムの中でも、データの記述については原信号と特徴抽出を行なった後の意味的構造とで大きく異なっているが、処理全体の記述は、信号処理が電子回路をその出発点としていることもあって、決まった機能を持つモジュールを接続したブロックダイアグラムの形で説明されている。特に、信号の復元・強調といった処理では、ディジタルフィルタをモジュールとするような記述が普通である。

データの流れを順にモジュールに通して処理するようなデータモデルとしてストリームがある。これは、扱う信号が1次元の時系列信号だけならば、ディジタルフィルタの基本要素 (Delay element, Multiplier) をそのまま表現することが可能である。また、ストリームで扱うことのできるデータ構造が線形リストのみであるが、その拡張である Series ではベクタやハッシュ表なども表現できるようになっている。

問題は、ストリームあるいは Series によって表現できるデータは、基本的に先頭から順番につまった構造を持つ必要があることである。信号データにディジタルフィルタを適用するだけなら問題はないが、クリッピングをおこなって特定の部分を取り出す操作を行なうと、値の定義されていない領域を表現する必要が出てくるため、ストリーミ的な考え方だけでは不十分といえる。

そこで、ストリームの考え方で提供されている基本操作を、ストリームではなく関数に適用することを考えると、信号のクリッピングされた部分は、関数の定義域外と考えることができ、信号処理を無理なく表現することができるようになる。このように考えると、処理のながれは関数の結合によってデータフロー的に表現されることになり、伝達関数のように、入力として与えられる値ではなく、関数そのものを中心としてシステムの振舞いを記述できるようになる。このような表現を計算機上のデータモデルでも可能にするため、カテゴリカル・コンビネータ理論 CCL (Categorical Combinatory Logic, 文献 [9]) の考え方を導入する。一般的なプログラム言語で、関数を組み合わせて新しい関数を定義するときは、関数の呼び出しを入れ子にして、

$$g(f(x))$$

のように記述するが、CCL では関数合成 $\circ$ が基本的な操作であり、

$$g \circ f$$

のように記述される。つまり前者が値の扱いを中心とした記述であるのに対して、後者は関数の扱いを中心とした記述となっており、またデータフローを $\circ$ によって表していると見ることもできる。

そこで本稿ではこの CCL をベースとして信号処理のデータモデルの記述を考えていくことにする。

3.3 信号上の領域

ここまで述べてきたことから、信号を表現する関数の定義域上の領域と、その構造を表現しなければならないことがわかる。領域とは部分集合のことであり、本節ではまず部分集合の表現について考える。

一般にプログラムで扱う際は単純に、組 $(start, end)$ といった形で区間を表現して扱うといった場合も多いが、本稿で扱おうとしているような不連続な部分を含むような領域を表現するためには十分ではない。

そもそも、集合は、ある要素をその集合が包含しているか判定する述語関数と、その集合に含まれる2つの要素が等しいか等しくないかを判定する述語関数によって定義される(細かい定義は後述)。そこで、本稿では集合もこの2つの関数の組によって定義することにする。ただ、これだけでは困るので、システムに組み込まれているアトミックな型を集合とし、システム手続きは関数として使えることとする。

このようにして、領域を定義することができたが、これだけではその領域内を掃引することができない。これは集合内の構造を定義していないからである。信号の持っている時系列の構造は、その信号を表現している関数の定義域、すなわち集合の構造によって定義されている。例えば、音声信号はその定義域が時間軸なので、ある時刻 t のサンプル点の“次の”のサンプル点とは、時間軸上における次の点 $t + \Delta t$ (Δt はサンプリング間隔) に対応するサンプルを指す。これが画像の場合ならば、定義域は2次元のピクセル座標である。

そこで集合の構造として、

- 集合操作のループへの展開 — 後者関数, 前者関数
- 集合の2要素間の演算 — 等価性, 四則, 比較

を定義する。

ここで特に重要なのは後者関数である。前節で、ストリームの考え方に基づいた基本操作を導入することを述べたが、これを実装するためには集合操作をループに展開する必要がある。基本操作をどのようなループに展開するかを決めるのは、関数の定義域であり、その順序構造にしたがってトラバースされることになる。

このような領域が持つ構造は、その領域が持つ性質の一種と考えられる。前に述べたように、特徴抽出とは信号のある領域の持つ性質を見つけ出し記述することであり、構造も含めて表現できる必要がある。このとき信号を表現している関数は、定義域の持つ性質を値域の持つ性質によって表現する、あるいは値域の持つ性質を定義域の持つ性質によって表現する、という使われ方をしているということができる。例として、上で挙げたような集合の構造的な性質や、線形変換、フーリエ変換などを挙げることができる。

3.4 領域の分割・記述と最適化

特徴抽出では信号上の領域に対して、意味を付加する。前節で関数の定義域上に領域を定義する方法について述べたので、そこに性質を付加し、それを利用する方法について述べることにする。

ある領域の持つ性質は、意味的構造を表現するグラフ中の、組(領域, 性質)を持つノードによって表現される。このとき、関数に対する操作の実現において、各部分ごとにわかっている性質を利用して、最適化をはかることができる。

これはオブジェクト指向の考え方の重要な柱である

- 自分自身の性質が記述されているようなデータ
- 動的なメソッド選択

と同じ考え方と見なすことが出来る。

信号処理とオブジェクト指向の関係についてであるが、確かにある1つの信号全体について記述すべき性質は少ない。しかし特徴抽出のように、その信号の一部分に注目すれば、“特徴”という性質を記述することができ、これを使ってメソッド選択を行なうことが出来るわけである。

しかし最初で述べたように、信号は非常に粒度が小さく量が多いデータであるため、1つ1つのデータについてメソッド選択を行なうような処理は望ましくない。そこで、信号を特徴を持った領域に分割して、それぞれの領域についてメソッド選択を行なうことを考える。また、1つの処理を行なうために複数の方法が考えられる時、どの方法を選ぶかをメソッド結合の延長として考えることにする。

4 モデル

ここでは、前節で述べた信号処理のデータモデルを一般化して、構造を持ったデータの集まりに対する処理を記述する方法について述べる。

4.1 集合

集合は、その定義より (文献 [7])

- 任意のオブジェクト x が集合 S に属するか属さないかを規定する
- 集合 S の任意の要素 x, y が等しいかどうかを規定する

という2つの機能を持つ。しかし、これだけではその集合に属している要素間にどのような関係があるか全くわからないので、要素間の関係の表現を導入する。

本稿では、ある集合の要素間の関係は、その集合の要素に対して適用可能な関数によって表現されと考える。例としては後者関数や距離が挙げられるだろう。このとき重要なのは、距離と言う概念が、いろいろな集合に対して定義できることからわかるように、要素間の関係を表現するこれらの関数の性質が、特定の集合に依存しないことである。つまり、これらの関数は対象とする集合毎にメソッドが定義されているような総称関数と捉えることができる。

このことから集合の構造とは、要素間の関係を表現するような総称関数の集まりであると考えることができ、各要素の値と切り離して考えることができる。逆に、構造を持たない値の集まりであるような集合と、集合の構造とを組み合わせることによって、構造を持った集合が定義されと考えることもできる

また、構造の特別な場合として、総称関数が閉じた演算子を表しており、その性質が公理によって定義されているような場合、数学的な構造を表現していると考えられる。

さて、集合の構造は総称関数の集合と定義したので、その包含関係によって集合の構造を階層化することができ、これをメソッド選択から見たクラス階層と考えることができる。このとき、ある構造 A のサブクラスであるような構造 B は、集合内の要素に対してより多くの総称関数が適用可能なので、構造 B に属するような集合は、構造 A に属す集合よりも、構造的な性質が“良くわかっている”と言うことができる。

4.2 関数

前に信号や信号に対する処理を関数として扱うという考え方を述べたが、ここでは本稿のモデルにおける関数を分類し、信号や信号処理を表現する関数の位置づけを明確にする。このとき、関数は2つの集合の要素間の関係を表現したものであるが、前節で集合を構造と値に分けることについて述べたので、関数の定義域や値域の持つ構造に注目して関数を分類する。

閉じた演算子 関数の定義域と値域が同じ集合であるようなもので、その集合の構造的な性質を表現していると見ることができる。集合の構造を表現する総称関数は、これらの関数をメソッドとして集め、抽象化したものと考えることができる。

良くわかっている集合への写像 集合の構造のクラス階層上で、関数の定義域の属している構造よりも、値域の属している構造のほうが、サブクラス側にあるようなもの、つまり、構造のあまり良くわからない集合から、構造の良くわかっている集合への関数である。例としては、組の属性値の参照 (射影)、述語関数、距離などが挙げられる。

良くわかっている集合からの写像 上とは逆に、構造の良くわかっている集合から、構造のあまり良くわからない集合への関数である。例としては、配列やリストのようなコレクション、サンプリングした原信号や、グラフにおけるノードからノードに付加された属性への対応などが挙げられる。

変換 関数から関数への対応づけを行なうような関数. 例としては, 関数合成 $\circ$, デジタルフィルタ, FFT などが挙げられるだろう.

この分類からわかるように, 信号処理のようなデータの集まりに対して処理を記述するためには, 関数から関数への写像である変換について考えなければならない. この時, 変換は関数を要素としてとり扱うため, このとき普通に関数を考える時のような値を中心とする考え方よりも, 関数それ自体を中心として扱うような考え方が望ましい. そこで, このような条件を満たすモデルとして CCL をとり入れることにし, CCL の基本関数を使って関数を組み合わせることによって, 変換を記述することを考えていくことにする.

4.3 変換

変換は, 入力として与えられる関数の定義域の構造や出力する関数の定義域の構造によって分類することができる. 以下, いろいろな変換のうち 1 次元の時系列データの処理に関係すると思われるものについて述べることにする.

4.3.1 順序構造を保存する変換

入力として与えられた関数を, 単に重複を許した値の集まりとみなし, その各要素について操作を適用するような変換を考える. すると, 変換して得られる出力の値は入力の値のみに依存し, 定義域上の位置, 例えば時系列データ $f[t]$ なら時刻 t には無関係に定まる.

この変換は, 基本的には入力の定義域と出力の定義域が同じになるので, 構造を保存する変換なのだが, 関数を適用する時エラーなどによって, 入力の定義域には含まれるが出力の定義域には含まれないような要素が存在する. そのためこのような変換は関数の定義域の順序構造を保存する変換となる.

本稿のモデルでは, このような変換としてマッピングとフィルタリングを考えている. これらの操作は, やはり信号などのデータフロー的な表現を考えているような ストリーム [1] や Series[2] で導入されているが, 入力の定義域が持つ構造が保存される点が異なっている.

マッピング

まずマッピングであるが, これは入力として与えられた関数 f の各点に, 関数 g を適用する変換で,

$$\text{map}(g, f) = g \circ f$$

とである.

適用例を挙げる. まず図 1 は原データ $f[t]$ を示したものである. これに $g(x) = x^2$ をマッピング, すなわち

$$\text{map}(x^2, f)$$

とすると, 図 2 となる.

しかしこのままでは入力信号 f 上の関数ではないので,

$$\text{Map}(g) : f \mapsto (g \circ f)$$

であるように CCL を使って map を書き直すと,

$$\text{Map} = \Lambda(\text{Compose})$$

となる. マッピングをこのように記述するのは, 信号 $f(t)$ に対して, $g_1(x)$ をマッピングしたのち, $g_2(x)$ をマッピングするというような操作が,

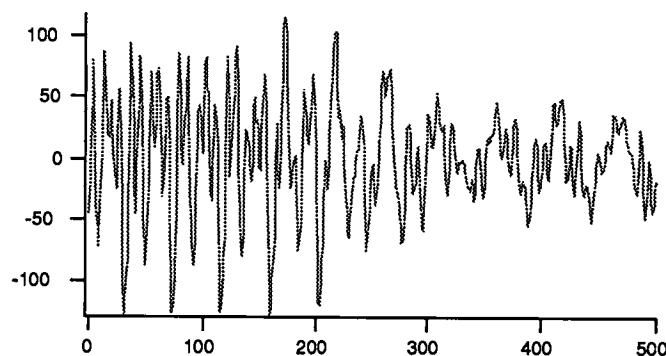


図 1: マッピングする前の原データ

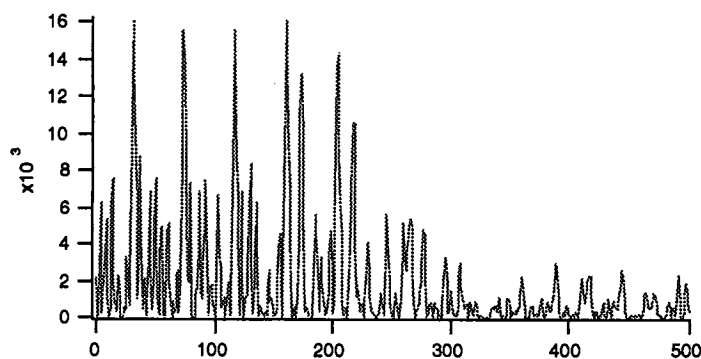


図 2: x^2 をマッピングした結果

$$Map(g_2) \circ Map(g_1)$$

という形で書き表せ、モジュールの結合によって処理を記述するのに向いているからである。

フィルタリング

フィルタリングは入力の変域の構造を変えずに、条件分岐を行なうような変換である。

$g = \text{filter}(p, f)$ は関数 f のうち述語関数 p を満たす部分を取り出した関数 g を出力する変換である。つまり f の定義域のうち p を満たさない部分は g の定義域に含まれない。

$(g_1, g_2) = \text{split}(p, f)$ は関数 f の定義域を述語関数 p によって分割した関数 g_1, g_2 を作り出すような変換である。

$$\text{filter}(F, G) = \text{Cond}(F, G, \text{Quote}(\perp))$$

$$\text{split}(F, G) = \langle \text{Cond}(F, G, \text{Quote}(\perp)), \text{Cond}(F, \text{Quote}(\perp), G) \rangle$$

ここでフィルタリングの例を挙げると、まず図 3 は原データ $s[t]$ を示したものである。これを $p(x) : x > c$ ($c = 30$) でフィルタリング、すなわち

$$\text{filter}(p, s)$$

とすると、図 4 となる。

ここで filter も CCL 風書き直すと、

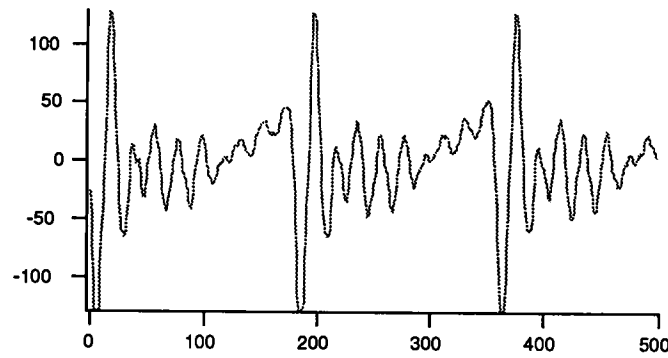


図 3: フィルタリングする前の原データ

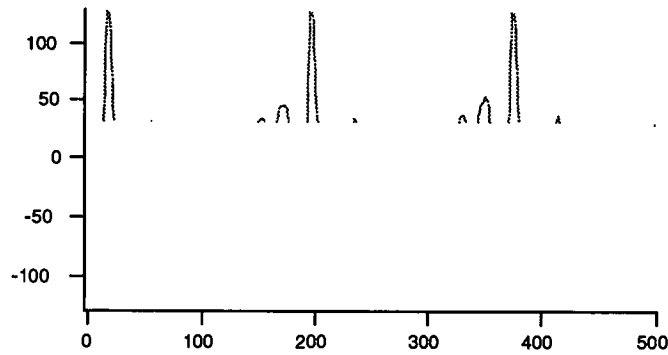


図 4: $x > c$ でフィルタリングした結果

$$Filter = App \circ \langle \Lambda(Cond \circ \langle Acc(2), Acc(0), Acc(1) \rangle), Quote(\perp) \rangle$$

となる。

4.3.2 列の変換

まず、ある関数が列であるとは、その関数の定義域が離散的で、かつ後者関数や前者関数が定義されるような構造を持つこととする。つまり関数上の注目している点とその前後の点の値を取り出すことができるような関数である。列の変換は、入力も出力も列であるような変換である。

このような変換の例としてデジタルフィルタが挙げられる。 z 変換によって表現される離散的な時系列信号は列と考えることができる。このときデジタルフィルタの基本的な構成要素の1つである遅延素子(delay element)は、

$$L\{f[n]\} = f[n-1]$$

であり、その伝達関数は

$$H(z) = z^{-1}$$

であるが、これを CCL を使って表現することを考える。

まず、入力の定義域上には前者関数 `pred` と後者関数 `succ` が与えられているものとする。すると入力を単位時間だけ遅らせる操作 `shifter` は、

$$\text{shifter}(f) = f \circ \text{pred}$$

のように定義することができる。

このように定義すると、デジタルフィルタの他の構成要素である乗算器やシステムの直列または並列な接続は順序関係を保存する変換なので、CCL を使った表現で書き直せることがわかる。ここで重要なのは、2つのデジタルフィルタ、すなわち変換を結合する場合で、CCL を使って記述した変換の結合が、伝達関数の合成と対応する点である。

構成要素	システム	伝達関数	変換	CCL による定義
乗算器	$L\{f[t]\} = af[t]$	$H(z) = a$	$\mathcal{L}(\text{Quote}(a)) : f \mapsto af$	$\mathcal{L} = \text{Map} \circ \langle \text{Id}, \Lambda(\Lambda(*)) \rangle$
遅延素子	$L\{f[t]\} = f[t-1]$	$H(z) = z^{-1}$	$\mathcal{L}(f) = f \circ \text{pred}$	$\mathcal{L} = \text{Compose} \circ \langle \text{pred}, \text{Id} \rangle$
直列な接続	$L_2\{L_1\{f[n]\}\}$	$H_2(z)H_1(z)$	$\mathcal{L}_2(\mathcal{L}_1(f))$	$\mathcal{L}_2 \circ \mathcal{L}_1$
並列な接続	$L_1\{f[n]\} + L_2\{f[n]\}$	$H_1(z) + H_2(z)$	$\mathcal{L}_1(f) + \mathcal{L}_2(f)$	$+ \circ \langle \mathcal{L}_1, \mathcal{L}_2 \rangle$

図 5: デジタルフィルタの構成要素

4.3.3 ベクタの変換

このベクタ (1 次元の配列) を入力としてとり、ベクタを出力するような変換。ベクタは列と構造的には同じであるが、各要素への参照が一定時間で行なうことができるようなものである。例としては、FFT などが挙げられるだろう。

4.3.4 集積関数への変換

ここでは、与えられた関数のある区間内での値の総和

$$f[n] \Rightarrow g(X) = \sum_{k \in X} f[k]$$

のような関数、すなわち与えられた関数上の区間について、その区間を要約した値を対応させるような関数を集積関数と呼ぶことにする。例として区間内の総和を挙げたが、他に最大値や平均値を求める関数や、零交差点の集合を求める関数が考えられるだろう。このとき、集積関数 g は与えられた関数 f の定義域上の領域に対して定義されるため、 g の定義域は f の定義域の部分集合の集合となる。

ここで考える変換は、入力として与えられた関数 f から、集積関数 g を作るような変換である。ただし信号処理の一般化なので、 f の定義域は離散的で、各要素をトラバースするための順序構造が定義されているものとしておく。

4.4 総称関数としての変換

ここまでいくつかの変換について述べたが、これらに共通する重要な特徴は、変換の入出力となる関数の値に依存せずに、変換のアルゴリズムを記述することが可能となっている点である。

中には、デジタルフィルタの乗算器のように、値をパラメータとして与えなければならないような変換もある。しかし、マッピングやフィルタリングのように、変換を生成するための関数を定義して、その入力としてパラメータを与えることによって、変換のアルゴリズムの表現を、特定の値に依存せず、関数の定義域や値域の構造を表現する総称関数だけを使う形で定義することは可能である。

また、上で述べた変換の分類を見ると、ある変換がどのような関数に対して適用できるかどうかは、入力となる関数の定義域や値域の構造、あるいは定義域と値域の対応関係によって決まると考えられる。

そこで、以前に集合と関数の関係について述べた時、集合を構造と値に分けて考え、その構造によって関数を分類したように、関数と変換について同じことを考えてみる。つまり、関数を構造的な性質と値に分けて考え、構造的な性質は、その関数の定義域や値域の構造や、関数に対して適用できる変換の集合によって定義することにする。すると、やはり関数の構造も総称関数の集合となり、その包含関係によってクラス階層を作ることができる。

変換のアルゴリズムは関数の定義域や値域上で定義された総称関数を使って表現されるが、一般に集合の持つ構造的な性質が多いほどアルゴリズムの最適化ができると思われる（上の分類でいえば列とベクタのような関係）。そこで変換も総称関数とし、入力される関数の構造に応じて変換アルゴリズムをメソッドとして、メソッド選択を行なうようにする。すると、データ構造の実装とアルゴリズムの記述とを独立させることができる。

5 実装の方針

多くの場合、ストリームは遅延評価を用いて実装される。確かにこれはデータ構造と計算プロセスを巧みに組み合わせたものといえるが、本稿のモデルのようにデータの持つ構造に拡張性を持たせた場合、計算プロセスのスケジューラとデータ構造がからんだ複雑なバグがおきやすく、保守性を下げてしまう。

そこで、データ構造とスケジューリングを分離するため、関数から関数への変換の実行を“スレッド (thread, 文献 [5])”として実現することにする。すると、関数は変換と変換を仲介するものであり、スレッド間の通信と考えられるので、関数を UNIX のファイルやパイプの概念を拡張した“バッファ”として実現することにする。

スレッドを使う理由は上で述べた他に、信号処理の性質、すなわち巨大なデータに対して計算処理を行わなければならないということが挙げられる。つまり、このような処理は非常に時間がかかるので、処理を構成する複数の変換をバッチ的に処理するのではなく、UNIX のパイプのように各変換を少しずつ並行に進めて、計算処理全体が終了するよりも早く、一部だけでも結果を得て、アルゴリズムやパラメータの変更を行ないたい、という要求がある。ただ、大抵の場合バッチ的に処理を進めた方が全体の計算時間は短くなるので、スケジューラを処理アルゴリズムと切り離して扱うことができることが望ましい。

また、信号処理に対する修正を行なう時などは、電子回路のように関連するモジュールの交換によって行なうことができると便利である。つまり、モデルのところでは、関数から関数への変換をモジュールとみなして結合できることを目標としたが、計算を実行する時も同様であるといえる。そこで、マクロ展開などによってまったく別のものになるのではなく、プロセスやスレッドのように実行中も操作することのできる実体を持つべきであろう。

パラメータやアルゴリズムの修正を行なった時、どのような部分に影響が及ぶか、あるいは要求駆動、すなわち必要とされる部分の値を計算するときには、関数と関数の依存関係を知る必要がある。本稿で述べたモデルでは、依存関係は変換によって決まる。そこで変換は、変更伝搬や要求駆動のイベントを伝えなければならない。これらはプロセス間通信のプリミティブによって実現することが可能である。また、デジタルフィルタの場合、伝達関数の係数を無視すると、依存関係の表現と見なすことが可能であり、依存関係を細かく計算することもできる。このように依存関係を知ることによって、“生きている”計算結果を得ることができる。

巨大なデータを扱うことを前提とするので、persistent object として扱わなければならないとは当然である。しかし、関数から関数への変換によって作り出されたこれらのデータを“生きている”状態に保つためには、その結果を得るための変換の連鎖そのものも persistent でなければならない。ここでは変換をスレッ

ドによって実現することを考えているので、必要とされているのは、実行している計算機をリブートしても実行を継続できるような persistent なスレッドである。

一般に信号処理のような計算量の多い処理は処理時間が長くなるので、計算機の不調でリブートすることがあると、その時点までの計算結果が無駄になりかねない。その意味で persistent なスレッドは障害復旧という点からも重要である。

以上のような理由から、変換をスレッドとして実行させるような実装が良いのではないと思われる。以下、スレッドとバッファを持つべき機能について述べておく。

スレッド

スレッドの実装は、Mach のように OS の機能としてスレッドを持っているものを使うか、Scheme のような continuation の機能を持つものを利用すれば簡単ではあるが、これらでは persistent なスレッドという要求を満たすことができない。そのため、現在設計中の実装は、CommonLisp でスレッドのエミュレーションを行なうことにしている。

バッファ

ある スレッド T_1 が出力として 関数 $f: X \rightarrow Y$ を生成し、それを スレッド T_2 が入力として受けとるとする。このとき バッファ $f_b: X_b \rightarrow Y_b$ は、 f の入出力の対応関係の一部を保持するような 関数である。 f_b の定義域 X_b は以下の 3 つの領域に分けられる。

$x \in X_b$ について、

- (1) $x \in X$ ならば $f_b(x) = f(x)$ (defined)
- (2) $x \notin X$ ならば $f_b(x) = \perp$ (undefined)
- (3) x が X に含まれるかどうか未確定 (unknown)

スレッド T_1 は write 操作で (3) へのみ書き込むことが許される (つまり append-only)。書き込まれた点は (1) あるいは (2) になる。スレッド T_2 は read 操作で (3) をアクセスした時その実行を block され、他のスレッドがスケジューされる。そして スレッド T_1 がその点に何か書き込んで (1), (2) になったところで unblock される。もし X が正の整数ならば、バッファは UNIX の ファイルとパイプを合わせたような性質を持っているといえる。

6 関連研究

信号処理のプログラミングに関しては、直接それを対象として議論している研究は少なく、通常は Fortran あるいは C で、配列やループを駆使して信号処理のプログラムを書いているのが現状であると考えられる。そこで、信号処理に関連すると思われるプログラミング・パラダイムをいくつか取り上げることにする。

6.1 構造化プログラミング

通常の信号処理プログラムは、Fortran や C によるプログラムで、信号を配列によって表現し、DO あるいは while, for といったループ構文を使って処理を記述する。従来から、構造化プログラミングやオブジェクト指向言語 (OOP) によって、データ構造の抽象化を行なうという考え方は広く受け入れられてきている。しかし、これらのモデルの提供するデータの抽象化は組 (tuple) の扱い方を中心に行なわれてきたといえる。

また、コレクションという形で集合も一般化され、iteration protocol など考えられているが、信号処理に実際に適用できるような抽象的なデータ構造は、相変わらず配列であり、これに対する操作は while, for などのループ構文か、基本的なループを一般化した程度に留まっている。

さらに、領域の扱いはユーザのプログラミングに任されていて、抽象化して取り扱う手段がないというのが現状と言える。

6.2 オブジェクト指向

信号の扱いを考える上では、オブジェクト指向の考え方に基づくデータモデルが、データが自分自身の持つ性質に関する情報を持っているという点で現在最も適していると考えられる。しかし、マルチメディア対応を大きな応用分野としてあげているオブジェクト指向データベースシステム (以下 OODB) にしても、今のところマルチメディアデータそのものは単なるバイナリデータの塊として扱っているものが多く、上記のような問題には対処していない。

その中で文献 [4] では、OODB に対する質問 (query) としてマッピングやフィルタリングを用いる考え方が示されている。また、複数の異なる等価性を導入しており、集合操作をおこなうときはどの等価性を利用するか指定する。

問題なのは、対象があくまでも通常のデータベースであるため、操作は組主体である。導入された等価性も、結局のところポインタを何書いた度って等価と見なすかというものであり、数値計算を含むような場合には不十分と言えるだろう。

6.3 関数型プログラミング

関数型言語で導入されたデータ構造であるストリームは、本稿のアイディアの説明をおこなうときにすでに少し触れたように、遅延評価によってデータ構造とスケジューリングを組み合わせて、数値計算をいくつかの決まったモジュールの組合せとして表現するものである。

特に関数型言語の FP や CCL に見られるように、関数の合成を使ったアルゴリズムの表現は、基本的なモジュールの結合として記述されるデジタルフィルタなどを表す上で有効であり、本稿のモデルも大きく影響されている。

しかし、ストリームの考え方自体は、いろいろな言語で取り上げられているが (FP[8], Scheme[1]), そのほとんどがデータ構造としてリスト構造を使っている。しかし、単純な線形リスト構造は信号の表現には向かない。

そこでリスト以外にも拡張したものとしては Series [2] を挙げられる。これはマクロパッケージとして公開されていて、ループの最適化など非常に強力なものだが、いくつか問題もある。リスト以外の構造も扱えるのだが、リストと同様にフィルタリングを行なうと条件を満たした要素が、データ構造の先頭から詰め込まれてしまい、構造が変化してしまう。また、あくまでマクロであり計算結果を求めることに主眼がおかれているため、パラメータを変更した時の部分的な再計算などは考慮されていない。

7 おわりに

本稿では、信号処理のプログラミングにおける問題点を分析し、このような粒度の細かい集合を柔軟に扱うのに適したデータモデルとその実装方法についていくつかのアイディアを述べた。

これらの考え方は、従来データベースや各種データモデルの対象外であった、粒度の細かいデータを取り扱うアプリケーションに、オブジェクト指向の考え方を導入するための出発点であると考えており、このような方向からの議論を進めることにより、現在、Fortran や C で記述されているこれらのプログラムを、もっと記述性や保守性の高いモデルを使って実装できるようになると考えている。

今後の課題としては、モデルによって表現できる処理の充実と、実行速度の向上が挙げられる。

モデルによって表現できる処理の充実とは、このモデルを使って実際に処理を記述するために、集合や関数の構造を分類しクラスライブラリを構築することと、それらに対して利用可能な変換を蓄積することである。

また実行速度については、今回述べたような実装方針の場合、まずコンテキストスイッチ (context switch) の速度と、スレッド間の通信のバッファリングが重要なのはもちろんだが、その他に、再帰的に定義された変換は、そのまま実行するとコンテキストスイッチを頻発させるため、実行速度を下げってしまうと考えられ、このあたりが課題になると思われる。

参考文献

- [1] Harold Abelson, Gerald Jay Sussman, and Julie Sussman. *Structure and Implementation of Computer Programs*. MIT Press, 1985. (邦訳は、マグロウヒルより、元吉文男訳, 1989).
- [2] Guy L. Steele Jr., editor. *Common Lisp The Language*. Digital Press, second edition, 1990. (邦訳は、共立出版より、井田昌之翻訳監修, 1991).
- [3] Azriel Rosenfeld and Avinash C. Kak. *Digital Picture Processing*. Academic Press, Inc., 1976. (邦訳は、近代科学社より、長尾 真 監訳, 長尾 真, 金出 武雄, 木戸出正継, 田村秀行, 松山 隆司, 1978).
- [4] Gail M. Shaw and Stanley B. Zdonik. A query algebra for object-oriented databases. In *Proceedings of the 6th International Conference on Data Engineering*, pp. 154–162, 1990.
- [5] Andrew S. Tanenbaum. *Modern Operating Systems*. Prentice-Hall International, 1992.
- [6] Apple Computer Eastern Research and Technology. *Dylan an object-oriented dynamic language*. Apple Computer, 1992.
- [7] 寺阪英孝 (編). 現代数学小事典, pp. 44–74. 講談社, 1977.
- [8] 富樫敦. 続 新しいプログラミングパラダイム, 第 5 章 プログラム代数と FP. 共立出版, 1990.
- [9] 横内寛文. 続 新しいプログラミングパラダイム, 第 6 章 カテゴリカル・プログラミング. 共立出版, 1990.

A CCL

ここでは本文中で導入した CCL の文法について述べておく。基本的には文献 [9] の拡張 CCL であるが、高階関数を扱うため、少し関数を追加してある。

まず関数を定義するための一般的な記法の準備をしておく。集合 A から集合 B への関数 f で各 $a \in A$ について $f(a) = b$ のとき、

$$f : A \rightarrow B, a \mapsto b$$

と書くことにする。また集合 A から集合 B への関数の集合を B^A と書くことにする。

CCL の構文規則は表 1 のようになっている。

$\langle \text{式} \rangle$	$::=$	$Quote(\langle \text{定数} \rangle)$
		$\langle \text{基本関数} \rangle$
		$Id \mid Fst \mid Snd \mid App$
		$\langle \text{式} \rangle \circ \langle \text{式} \rangle$
		$\langle \langle \text{式} \rangle, \langle \text{式} \rangle \rangle$
		$\Lambda(\langle \text{式} \rangle)$
		$Fix(\langle \text{式} \rangle)$
		$Cond(\langle \text{式} \rangle, \langle \text{式} \rangle, \langle \text{式} \rangle)$

表 1: CCL の構文規則

以下、それぞれの式の意味について述べる。

CCL においてもっとも基本となる操作は、関数合成 $\circ$ であり、

$$f_1 : A \rightarrow B, a \mapsto b$$

$$f_2 : B \rightarrow C, b \mapsto c$$

のとき、

$$f_2 \circ f_1 : A \rightarrow C, a \mapsto c$$

である。

2 個以上の引数をとる関数、例えば

$$f(a, b)$$

は、組 (a, b) という引数を 1 つとる関数と考える。すると、2 引数以上の関数の定義域は直積を使って表現することができ、関数 f の第 1 引数が A 上を動き、第 2 引数が B 上を動くとき、 f の定義域は $A \times B$ となる。これらは関数の返り値についても同様に、多値関数も組を使って表現することにする。

また、直積に関する基本操作を行なう関数として、

$$\pi_1^{X,Y} : X \times Y \rightarrow X, (x, y) \mapsto x$$

$$\pi_2^{X,Y} : X \times Y \rightarrow Y, (x, y) \mapsto y$$

を定義する。これらの関数は射影 (projection) と呼ばれている。($\pi_1^{X,Y}, \pi_2^{X,Y}$ をそれぞれ $Fst^{X,Y}, Snd^{X,Y}$ と書くこともある。)

これを使って次のような操作を定義する。

- 関数作用 (apply) :

$$\begin{aligned} App^{A,B} : B^A \times A &\rightarrow B, \\ (f, a) &\mapsto f(a) \end{aligned}$$

- 関数組 : 定義域が同じ 2 つの関数 $f_1 : Z \rightarrow X$ と $f_2 : Z \rightarrow Y$ について,

$$\begin{aligned} \langle f_1, f_2 \rangle : Z &\rightarrow X \times Y \\ \langle f_1, f_2 \rangle(z) &= (f_1(z), f_2(z)) \end{aligned}$$

- カリー化 : $\Lambda(F)$ は $F : C \times A \rightarrow B$ をカリー化したもので,

$$\begin{aligned} \Lambda(F) : C &\rightarrow B^A, c \mapsto k_c \\ k_c : A &\rightarrow B, a \mapsto F(c, a) \end{aligned}$$

- 定数化 :

$$Quote(c) : \forall x \mapsto c$$

- 条件分岐 :

$$Cond(F, G_1, G_2) = \begin{cases} G_1 & F : X_1 \rightarrow Bool, x \mapsto true \\ G_2 & F : X_2 \rightarrow Bool, x \mapsto false \end{cases}$$

- 不動点 :

$$Fix(F) : A^{C \times A} \rightarrow A^C$$

$$f$$

ただし

$$F \circ \langle Id, f \rangle = f$$

これらについて表 2 のような公理が定義できる. この他に処理系によって提供される基本関数があるが, これはモデルごとに異なる.

ここまで述べた定義を使うことによって, CCL はラムダ計算の別の表現になっていることが知られている (文献 [9]). つまり, 一般のプログラム言語で使われる式は, ここまで述べた定義を使って関数によって表現できるわけである.

一般の式から CCL への変換は, 関数型言語で定義された関数を, データフロー図に変換することに似ている. その場合関数を使った表現における $\circ$ が, データフロー図でフローを表現している矢印に相当するが, 引数の順序の関係で矢印が交差するような場合は, 射影 π_k や恒等写像 id を使うことになる.

ここで, 文献 [9] の CCL を少し拡張する.

まず記述を簡潔にするための関数を以下のように定義する.

- 関数合成 : $F : A \rightarrow B, G : B \rightarrow C$ のとき $Compose^{F,G} : A \rightarrow C, (f, g) \mapsto g \circ f$

- カリー化 :

$$\begin{aligned} \Lambda_2(F) : C &\rightarrow B^A, c \mapsto k_c \\ k_c : A &\rightarrow B, a \mapsto F(a, c) \end{aligned}$$

$(F \circ G) \circ H$	$=$	$F \circ (G \circ H)$
$Id \circ F$	$=$	F
$F \circ Id$	$=$	F
$Fst \circ \langle F, G \rangle$	$=$	F
$Snd \circ \langle F, G \rangle$	$=$	G
$\langle F, G \rangle \circ H$	$=$	$\langle F \circ H, G \circ H \rangle$
$\Lambda(F) \circ H$	$=$	$\Lambda(F \circ \langle H \circ Fst, Snd \rangle)$
$App \circ \langle \Lambda(F), G \rangle$	$=$	$F \circ \langle Id, G \rangle$
$Quote(c) \circ H$	$=$	$Quote(c)$
$Cond(F, G_1, G_2) \circ H$	$=$	$Cond(F \circ H, G_1 \circ H, G_2 \circ H)$
$Fix(F) \circ H$	$=$	$Fix(F \circ \langle H \circ Fst, Snd \rangle)$
$Fix(F)$	$=$	$F \circ \langle Id, Fix(F) \rangle$

表 2: CCL の公理

• 射影:

$$Acc(n) = Snd \circ \underbrace{Fst \circ \dots \circ Fst}_{n \text{ 個}}$$

これらは高階関数を定義する時に必要となる。

次に、エラーを表現するための特別な値として $\perp$ を導入する。関数 f が $f: A \rightarrow B$ のとき、

$$\begin{cases} a \mapsto b & (a \in A) \\ a \mapsto \perp & (a \notin A) \end{cases}$$

とし、定義領域外の値を関数に適用したことを表す。また基本的に関数は、

$$f(\perp) = \perp$$

でなければならない。 $\perp$ が定義域に含まれる例外的な関数としては、CommonLisp[2] や Dylan[6] などの言語に見られるようなコンディションシステム (condition system) が挙げられる。

適応的プログラム開発環境に向けて

安賀 広幸 下條 真司* 西尾 章治郎** 宮原 秀夫

大阪大学基礎工学部情報工学科

*大阪大学大型計算機センター

**大阪大学情報処理教育センター

1. 現状の問題点

プログラムは、それを実行するシステムの変化やユーザの要求の変化に対して、不安定にならない "強さ" と、できるだけプログラムの変更をしないですむような柔軟性が望まれる。これらシステムの変化としては、プログラムが実行される環境を構成しているCPUやディスク、ネットワークといったハードウェア、OSやライブラリやユーティリティのようなソフトウェアの構成の変化があげられる。また、利用者から見たプログラムの意味、つまり本質的なアルゴリズムは変化しないが、精度、処理時間、コストなどの要求が変化するという場合も多い。この場合、これらの要求に応じて、CPU、メモリ、ディスクといったものの使い方をチューニングする必要がある、それがプログラムの変更を招く。例えば、計算機のメモリー量に応じて行列の表現方法を変えるなどである。例えば、精度や速度の要求によって、細かいアルゴリズムやコンパイラのオブティマイズレベルは変り、コストの要求からファイルのインデックス作成やクラスタリング、ネットワークの使い方などが変る。また、大量の実験データ処理を行う場合など、前もって概算値を求めてそれを見て、計算の精度を細かくするとか、重要な部分だけを細かく計算するとか、といったことを行なう必要が出てくる。

これら外部からの要求の変化に対して、従来は個々の要求に応じてプログラムを変更することで対処していた。プログラムの変更を最小限にとどめるためにモジュールに分割するという方法が一般的である。つまりモジュール間のインターフェースを定め、モジュール内の実現を隠し、モジュール単位での交換を可能にすることで、ダイナミックな変化に対応するという考え方であり、オブジェクト指向でいうカプセル化もこの概念の延長線上にある。しかし、上記の外部の要求の変化は、通常モジュールの分割そのものを変えてしまうため、プログラムの大幅な変更を招くことになる。つまり、これらの変化に対処するためにはより柔軟なモジュール分割の方法とさらに分割を動的に変えられるような仕掛けが必要となる。

このような場合、ユーザの要求に対して、どのようなチューニングのテクニックを組み合わせるべきか決めるための統一的な機構があれば、全体としてのアルゴリズムと外部の要求に応じた個別のテクニックを分離できる。その場合、まず処理したいデータ全体の集合を考え、それに適用するアルゴリズムを記述する。個別のテクニックは、データの部分集合ごとにその特性にあったメソッドを書くことによって取り扱うことになる。そうするとユーザの要求が変化すれば部分集合の取り出し方や、適用するメソッドが変えて対応することが出来る。

この集合の取り出し方をクラスに対応させると、アルゴリズムを記述するための抽象的なクラスをスーパークラスに作り、個別のテクニックはサブクラス側に作ることになる。しかし、個別のテクニックもアルゴリズムとは別のクラス階層を持つことがあるなど、1つのクラス階層の

中にうまくまとめられないことも多いし、ユーザの要求が変化するため、クラスよりも柔軟な集合表現が必要になってくる。ところが現在のオブジェクト指向で対象に応じてメソッドを選ぶ枠組みは、クラスかインスタンスしかないため、ユーザの要求にダイナミックに応じることはむずかしい。

この強さや柔軟性を持つプログラムのことを適応的(adaptive)なプログラムと呼ぶことにする。本稿では以上のような問題点に対処するため、適応的プログラムを書くための枠組として、Region機構を提案する。Region 機構では、個々のインスタンスとクラスの間、インスタンスのグループや領域といったものを表現し、それに対する処理を記述する。

2. Region 機構の考え方

Region機構の基本となる考え方は次の2つである。

1. インスタンスから集合を取り出す(Projection)
2. 集合に対して総称関数を適用して集合を得る

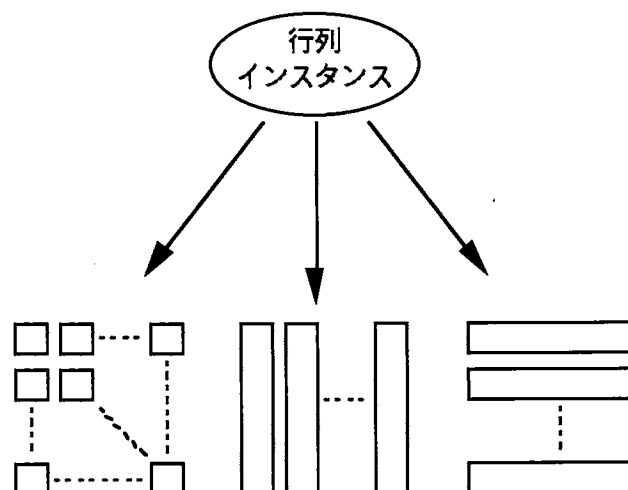
以下それぞれについて説明する。

2.1 Projection

Projection とは、一般的なオブジェクト指向におけるインスタンスの属性へのアクセスの拡張である。ただ集合とインスタンスとの関係には、集合をいくつか集めて1つのインスタンスとすること(aggrigation)と、1つのインスタンスが目的に応じて集合をとりだすことの2つがあり、通常組み合わせて使われるが、C++などにおけるインスタンスの属性では、視点を動的に切り替えることが出来ず、この関係をうまく表現できない。そこでどちらの場合についても、インスタンスに対して視点を決めることによって集合を取り出すと考える。この視点を決めることをProjectionと呼ぶ。実際には各視点に対するProjectionはアクセス関数という形で定義する。

ここでデータ処理の例として、画像や音声のデータをベクタや行列で表現する場合を挙げる。

通常行列は行と列を指定して各要素の値へアクセスをするが、行ベクタの列ベクタ、あるいは列ベクタの行ベクタと考えてアクセスしたいこともある。もしこの行列インスタンスが画像データを表現しているのなら、行列をジグザグスキャンをして出来るベクタと考えてアクセスする場合もある。また各要素の値に関数やフィルタを適用して取り出すことも考えられる。



インターフェースとインプリメンテーションを分離することは、構造化プログラミングやオブジェクト指向プログラミングの利点としてよく述べられているが、ここでは集合へのアクセスのインターフェースを、アクセス関数の束として決めることで、単に計算機内の内部表現を隠すことのほかに、集合の構造的な意味の定義をおこなっていると考えている。

例えばベクタのインスタンスを、行ベクタと見るか列ベクタとして見るかは、内部表現やアクセス関数の実装のほとんどの部分に影響しないが、全く別のものを表現していると考えなければならない。すなわち行ベクタと列ベクタは足せないからである。おなじことが行列にも言える。行列インスタンスを、行列、行ベクタの列ベクタ、列ベクタの行ベクタのいずれと見るかによって、掛け算のどのメソッドが選ばれるからである。

しかし、行列演算で転置がよくもちいられるように、Region機構においてもこれらの視点の間の相互変換(cast, coerce)が行える。特に、長さ n の行ベクタあるいは列ベクタと、 $(n, 1)$ 行列あるいは $(1, n)$ の行列の間の相互変換は有用であろう。

これらの構造を区別すると、データの処理が一見うまく動いているようなのだが、転置し忘れていて意味のない計算をすることを避けることができる。

2.2 集合に対する処理の分類

インスタンスから集合が取り出されたら、次はその集合に処理を適用し、結果として集合を得る。これは、UNIXなどのストリームの考え方に似たところがあるので、まずその性質を考えてみる。

UNIXなどのテキストストリームの考え方は次の3つの側面を持っている。

- (1) フィルタをパイプで連結して、入力となるファイルに適用し、結果をファイルとして得る。各フィルタは、入力ストリームからデータを読み、処理結果を出力ストリームに書き出す。

- (2) ストリームを読み書きする単位は文字である。
- (3) ファイルは、先頭から文字が詰まっており、アクセスは先頭から順に読み書きすることで行う。

(1)の性質によって、複雑な処理をフィルタをいくつかパイプでつなぎあわせて表現することができ、処理の流れを把握し易い。これは、入力に変更があったとき処理した結果を更新する変更の伝搬(change propagation)を行うとき、データに対する処理の流れが依存関係の記述として使うことができる。そこで、Region機構では、上で述べた性質のうち(1)の考え方を受け継ぎ、(2)と(3)を拡張して入出力にオブジェクトの集合を取り扱えるようにする。

まず(2)の拡張として、入出力の単位を、文字からオブジェクトかオブジェクトの集合の部分集合とする。部分集合を入出力の単位とするのは、実数上の区間 $[a, b]$ といった含まれる要素に1つ1つに処理を適用できないものを扱うことを考えるからである。

次に(3)の拡張として、先頭から各要素が順に列んでいるような構造だけでなく、ベクタや配列、あるいはテーブルのようなものも扱うことにする。

最後に、フィルタにオブジェクト指向の考え方を拡張して、集合に対する総称関数と考える。つまり、入力として与えられた集合が、性質のことなるいくつかの部分を持っているとき、各部分ごとにその性質に応じて適用する処理すなわちメソッドを変えるわけである。これはクラスとインスタンスの中間にあるようなインスタンスのグループやクラスの部分集合のようなものに対する、メソッドの書き方といえるだろう。

3. Region機構に必要な性質

3.1 集合の構造

前章で述べたような集合に対する総称関数をつくるためには、もとの集合を条件によって部分集合に分割し、処理を行った後一つの集合にまとめることができればならない。このとき、もとの集合を分割せずに処理を行う場合と同じ結果を得るためには、集合の分割/併合と集合の構造と処理の関係を考えなければならない。

まずもっとも単純な場合として、処理を行っても集合の構造が変化しない場合を考える。

構造を変えない基本的な操作としてマッピングとフィルタリングの2種類に考えることにする。

マッピング	集合の各要素に関数を適用する操作 (例: 画像データの明るさやコントラスト、色調などの変更)
フィルタリング	集合の各要素について条件を満たすものからなる部分集合を取り出す操作

(例:画像データのうちある明るさ以上の点を取り出す, ファイル中のある文字列を含む行を取り出す)

この時、フィルタリングを行っても、もとの集合の構造に変化を起こさないようにするために、集合は値が未定義になっている要素を持つことが出来る必要がある。これにあわせて、マッピングは集合の要素の値が未定義のものについては、結果も値が未定義とする。

これを使って総称関数を定義する場合は、集合の分割をフィルタリングで行い、併合はマッピングの特殊な場合として扱うことが出来る。つまり集合の分割が disjoint であり、各部分集合に適用する操作も構造を変えないならば、併合は部分集合の処理結果の中から値が定義されているものを選ぶだけなので、このルールをマッピングすればよい。

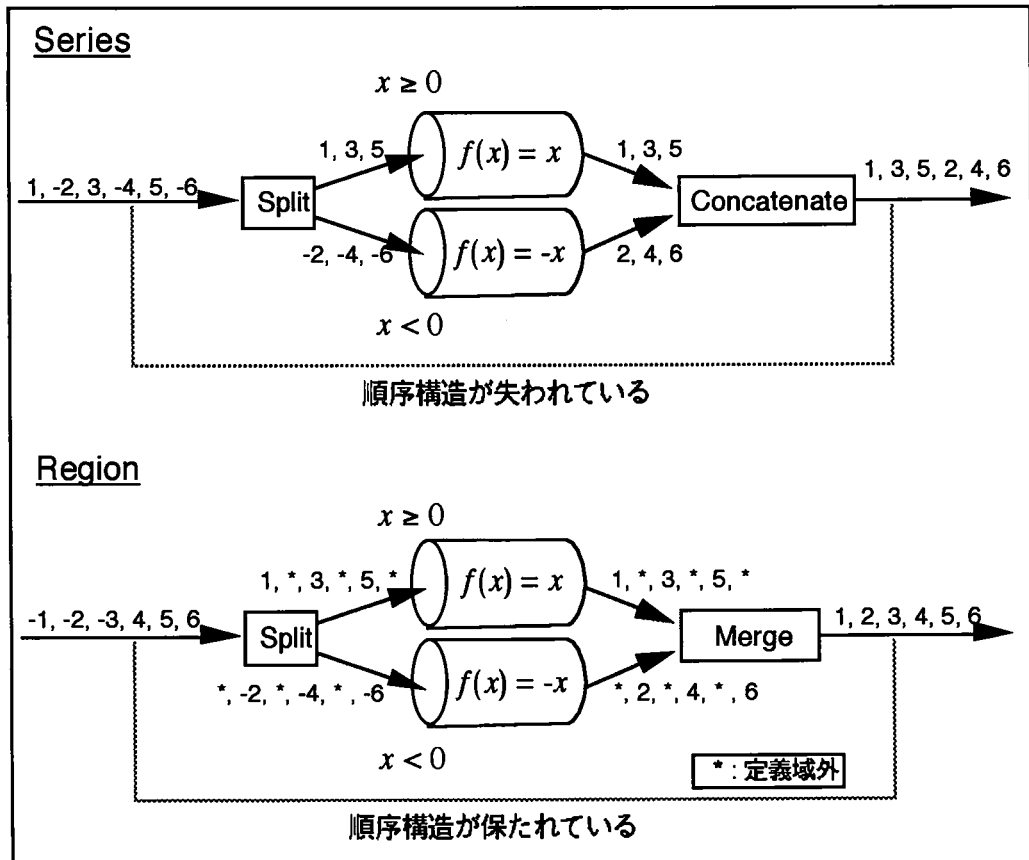
集合の構造として順序列を選んだ場合、Series との違いはこの併合の違いと言ってよい。簡単な例を図に示す。これは、順序列の各要素の絶対値を求めている例で、各要素の値が正か負かによって集合を split し、それぞれにマッピングを適用してからマージしている。このような操作は、集合の各要素を先頭から詰め込んでしまうようなストリームではうまく扱えない。もとの構造を復元するようなマージができないからである。

Series:

```
(let (x1 x2 y1 y2)
  (multiple-value-setq (x1 x2)
    (split (scan '(1 -2 3 -4 5 -6)) #'plusp))
  (setq y1 (map-fn t #'(lambda (x) x) x1)
        y2 (map-fn t #'(lambda (x) (- x)) x2))
  (collect (concatenate y1 y2)))
==> (1 3 5 2 4 6)
```

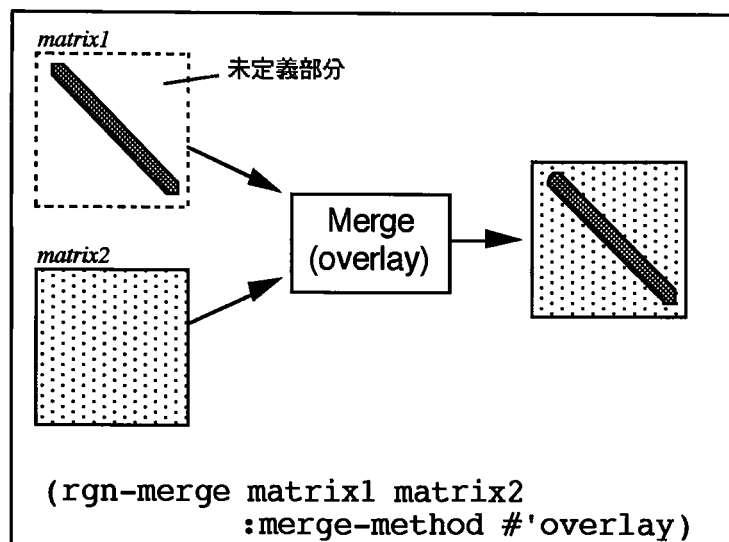
Region:

```
(let (x1 x2 y1 y2)
  (multiple-value-setq (x1 x2)
    (split (rgn-scan-list '(1 -2 3 -4 5 -6)) #'plusp))
  (setq y1 (rgn-map-fn t #'(lambda (x) x) x1)
        y2 (rgn-map-fn t #'(lambda (x) (- x)) x2))
  (rgn-collect (rgn-merge y1 y2)))
==> (1 2 3 4 5 6)
```



図? Series と Region の違い

処理によって集合の構造が変化する場合は問題がむずかしくなる。この場合も集合に関する総称関数はフィルタリングとマージで実現するが、マージするときに部分集合の処理結果が重なりあう部分ができってしまう。このとき併合するためのルールはいろいろ考えることができるので、適切なもののプログラムを記述するときに選ぶ必要がある。併合のルールの例としては、優先度の高い集合から各要素の値が定義されているか見て定義されている値を返すオーバーレイが代表的だが、他にも足算, 平均, 大小関係 などを使うこともできる。



3.2 遅延評価

上で述べたような集合の処理を、そのまま実際に実行すると、大きな集合を扱う場合にはCPUもメモリも大量に必要になってしまいます。しかし、実際に必要な処理結果が処理結果として出てくる集合の一部だけであれば、結果が本当に必要な部分についてだけ処理を行えばよい。そこで遅延評価を使う。これは処理をはじめた時点では、結果を得るためのデータの処理の流れだけを記述したインスタンスを作り、実際に値が必要になるまでは実際の計算をおこなわないようにする方法である。

実装方針としては、欲しい結果1つ1つについて生成方法を記述するのは効率が悪いので、集合を生成方法の記述に使っている。実際に処理を実行するときには、入力として与えられている集合のうち、処理の実行に必要な部分集合を求める。この処理に必要な部分集合をどれぐらい小さく押えるかは、つまり遅延評価の効果の大きさは、実行する処理が集合の構造を変えるか、変えるならばどれほどか、による。

ここで、実際に処理を実行すべき集合の大きさを、うまく決めることが出来るかどうかによって、処理の分類が出来る。マッピングやフィルタリングのような構造を変えない処理の場合は問題ないが、構造を変えてしまうような処理の場合はむずかしくなる。これもまた2つの場合に分けることが出来る。

一方は、処理を実行すべき範囲が集合の値には依存しない操作で、処理のアルゴリズムを書くときに、あわせて処理を実行すべき範囲を計算するメソッドを書くことが出来る。例としては、ベクタの移動平均とか、画像データに対するガウシアンオペレータを考えることが出来る。

もう一方は、処理を実行すべき範囲が集合の値には依存してしまう操作で、実際に値を入力して実行するまで必要な部分が判らないため、遅延評価が有効ではない。簡単な例としては、行列式やFFTが挙げられる。このような操作を大きな集合に適用すると、再計算の量が増えてしまい問題がおこる。Region 機構で採ろうと考えているこの問題の解決方法は次の2つである。

集合の持つ性質を調べる

行列の積を求める場合、一方の行列が特殊な行列、例えば対角行列になっている場合は、もう一方の行列の値が変更されても、結果に影響する範囲を小さくすることが出来る。これは変更の伝搬にともなう再計算の量を減らすことにもつながる。

ただし集合が再計算の量を減らすのに役立つような性質を持っているかどうかの判定にもコストがかかるので、集合の性質を調べるのに必要なコストと、そのまま再計算をするコストとのトレードオフになる。

許される誤差の範囲を決めて計算量を減らす

演算精度をdouble から float におとすとか、画像データのピクセルを間引いて画像処理おこなったり、CGをレイトレーシングではなくポリゴンでレンダリングしたりすることが例として挙げられるだろう。

この方法は、もとの集合に変更があった場合だけでなく、処理の実行一般に適用できる。つまり、誤差の範囲を緩めて概算値を求めておき、注目する部分についてのみ細かく計算するわけである。このようなテクニック自体は別段目新しいものではないが、Region機構では処理そのものの流れと、誤差の大きさをコントロールするテクニックとを独立に記述し、統一的なチューニングの方法を提供することを目的としている。

遅延評価を実行するときの、実際処理すべき部分集合を求める作業を、逆方向から行うのが変更の伝搬である。1箇所の値を変更したとき、その影響がひろがる範囲の大きければ、イベントの爆発がおきてしまい、実行不可能になってしまう。遅延評価もこのイベントの爆発を抑える働きがあるが、それでも十分でないときは、上で述べたような、許容誤差を緩くする方法が有効ではないかと考えている。

処理の実行に必要な部分集合をどの程度細かく求めるかも重要である。集合の要素が大きなデータである場合や時間のかかる処理を実行する場合は、処理すべき要素を細かく求める方がよいが、集合の要素が小さなデータである場合や簡単な処理を実行する場合は、必要以上に処理を実行することになっても、一度にまとめて実行するほうが効率がよい。このことから遅延評価の粒度がチューニングのパラメータであり、これはユーザが必要に応じてダイナミックに変えることができる。

3.3 変更とバージョン

変更の方法

これまで集合に対する処理について書いてきたが、ここでは集合の変更方法について考えてみる。ここで問題なのは、変更の伝搬を行うことを考えているとき、変更して得られる結果でもとの集合を書き換えてしまい、集合に対して破壊的代入による変更を行うと、いろいろと意味的な問題が起こる。そこでRegion機構では集合に対する変更は、変更して出来た結果を新しいバージョンとして取り扱い、破壊的変更は行わない。こうすると意味的な問題は避けられるが、バー

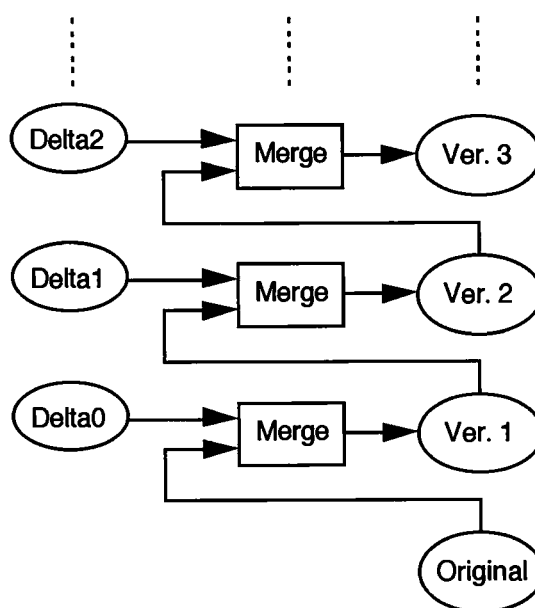
ジョン管理のための機構が必要になる。

変更は新しいバージョンの生成という形で行われるため、変更には集合に対する処理全てを使うことが出来る。もっとも基本的な変更は、もとの集合と変更する部分をマージすることである。このときマージの規則としてはオーバーレイが普通だろう。これは、変更された後の最新の値を常に参照するようにすると、代入が行われたように見える。

テキストエディタによるファイルの修正のような場合は、いろいろな処理を複合して新しいバージョンが生成される。しかし、エディット中のキーストロークやマウスの動きを全て再現する必要がなければ、もとのバージョンと新しいバージョンの差分をとってマージしたと考えてもよい。

バージョン機構

ここでRegion機構におけるバージョン機構について考えてみる。まず、マージによって作り出されるバージョンを集めたバージョン集合を考える。話を簡単にするために、常に変更は最新の物のみについて行われることとする(線型バージョン)。このときバージョン集合は順序列としての構造を持っていて、Region機構による処理の対象になり得る。つまりバージョン集合とは、もとの集合と変更差分の列に対してマッピングを行った結果であると考えられるわけである。



図?

変更の伝搬は、バージョン集合に対する処理という形で実現できる。依存関係すなわちバージョン集合に対する処理は、マッピングやフィルタリングであると考えられる。

例えば、`foo.lisp` をコンパイルして `foo.fasl` を作ることにする。`foo.lisp` のバージョン集合を `foo-lisp-vers`, `foo-fasl-vers` とすると、

```
(setf foo-fasl-vers
      (rgn-map-fn t #'compile-file foo-lisp-vers))
```

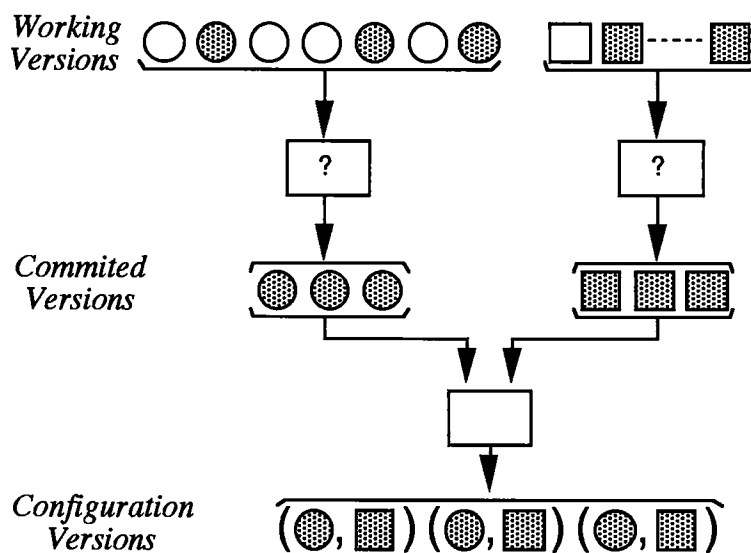
と書ける。これは、

```
(let* ((foo '(1 2 3 4 5)))
  (rgn-collect (rgn-map-fn t #'(lambda (x) (+ x x)) foo)))
==> (2 4 6 8 10)
```

と処理の流れ自体は同じである。ただし、Region機構が動いている間、バージョン集合は大きくなり続ける可能性があるので、インクリメンタルな処理が必要である。

また、複数のバージョン集合からの入力処理して結果を得るような場合は、入力の組み合わせ(コンフィグレーション)をバージョン集合として持たなくてはならない。このときちょっとした修正によって、もともになる各バージョン集合に新しいバージョンが出来るたびに、コンフィグレーションの新しいバージョンが出来たのではたまらない。

そこで、ある程度修正の句切りのついたところでcommitをおこなって、個々の作業中のバージョン集合(Working Version)から、コンフィグレーションに反映されるバージョンの集合(Committed Version)を作る。そして、このコミットされたバージョンの集合から、コンフィグレーションのバージョン集合を作る。



図?

変更伝搬

Region機構におけるもっとも基本的な使い方は、バージョンの再構成である。つまりバージョン管理機構は、すべてのバージョンの値を持っているわけではなく、差分データと変更処理の履歴しか持っていない。そして実際にあるバージョンが必要になった時点でその値を再構成する。これは変更が繰り返し行われた後、処理を実行するような場合に有効に働く。

3.5 キャッシュ

集合の値のキャッシュ

一般的な意味でのキャッシュで、処理を実行したときの中間結果をキャッシュすることによって、再利用される場合の性能を改善できる。このキャッシュはバージョン管理との一貫性を変更の伝搬によって一貫性を保つことができる。キャッシュの大きさもダイナミックに変えることの出来るチューニングパラメータである。

性質判定のキャッシュ

集合の各要素が、最初から条件を満たすように作った集合や、フィルタリングによって条件を満たすようになった集合は、その条件を満たしていることをキャッシュする。これを利用して、マッピングが(c)の場合でも、(b)のように再計算すべき領域を短時間で計算できる場合がある

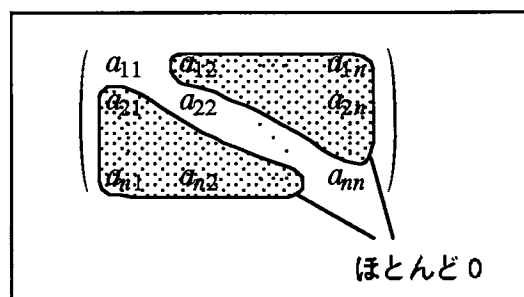
4.Region の応用

4.1 ストレージの階層化

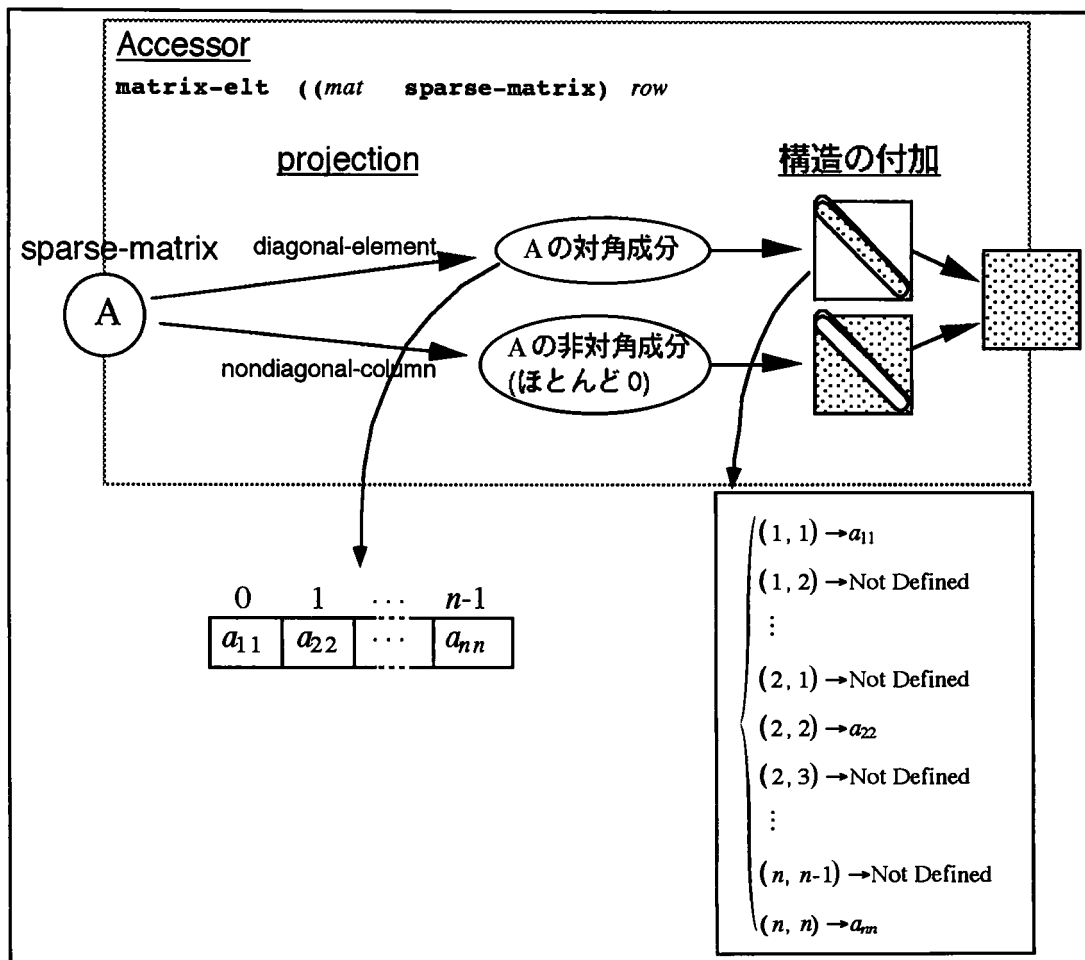
集合のインプリメントにRegion機構を使うことも考えられる。例として、対角成分以外がほとんど0であるようなスパース行列をあげる。この場合、対角成分と非対角成分を別々に持っておき、2つをマージして1つの行列として外部に見えるようにする。非対角成分はほとんどが0であることを利用して、メモリの使用量が少ない内部表現を使うことが出来る。

このような内部表現は、行列全体を掃引するような処理に対してはアクセスの速度を落してしまうが、行列式の概算値を求めるときなどは、許される誤差の制限を緩くして非対角成分はすべて0を返すようにすることができる。このような場合は、非対角成分をディスク上に追い出すことが出来る。

このような考え方は、大きな画像や文書のアーカイブなどにも適用できる。文書の場合は、文書のテキスト自身は圧縮しておき、検索用のインデックスファイルのみメモリ上にキャッシュするようなことができる。これはアーカイブがリムーバブルなメディアやネットワークの向こうにあるような場合の検索に有効である。



図? 対角成分以外はほとんど0のスパース行列



図? sparse-matrix クラスのオブジェクトへのアクセス

4.2 いらない領域の整理

例えば、

コミットされたバージョンを遅延評価を使って再構成するために必要でないオブジェクトは、不用なので整理する(Working Version の整理)

ここ半年の間にアクセスされていれば、再構成のためには不用でも、残しておく(ハイパーテキストへの手がかり, メモ, ...)

再構成手続きの圧縮

テキストファイルの修正差分として、1時間前のキーストロークが残っているのはいいが、1カ月前のものは不用。修正前のファイルと修正後のファイルの diff だけとって修正手続きを直しておけばよい

などのルールを使って不用なデータの整理をすることができる。

4.3 電子メール

Region 機構を、信号処理以外に適用する例として、電子メールを考えてみる。

他人とメールのやりとりをしていると、メールの返事を書くときに、もとのメールを引用する。その場合、現状ではもとのメールの一部をコピーするためもとの context を失ってしまい、あとから議論の流れをたどるとか、トピックを検索するといったことが、非常に行いにくい。

そこでRegion 機構を利用したメールシステムを構築することにより、メールの本文中の引用部分や自分で書いた部分を、それぞれ region として取り出せるようにし、メールの構造化を行うことができる。引用の場合は、その部分をどこからコピーしたかを、処理の流れとして記録できるので、処理の流れをリンクとみなして議論の流れをたどることや、話題の分岐などたどって、話題の間の関連を見ることが出来る。

5. 現状と問題点

現在はCLOS を使って実装中であり、現在はCLOS を使って実装中であり、順序列を遅延評価を使って扱うようなプロトタイプができている。

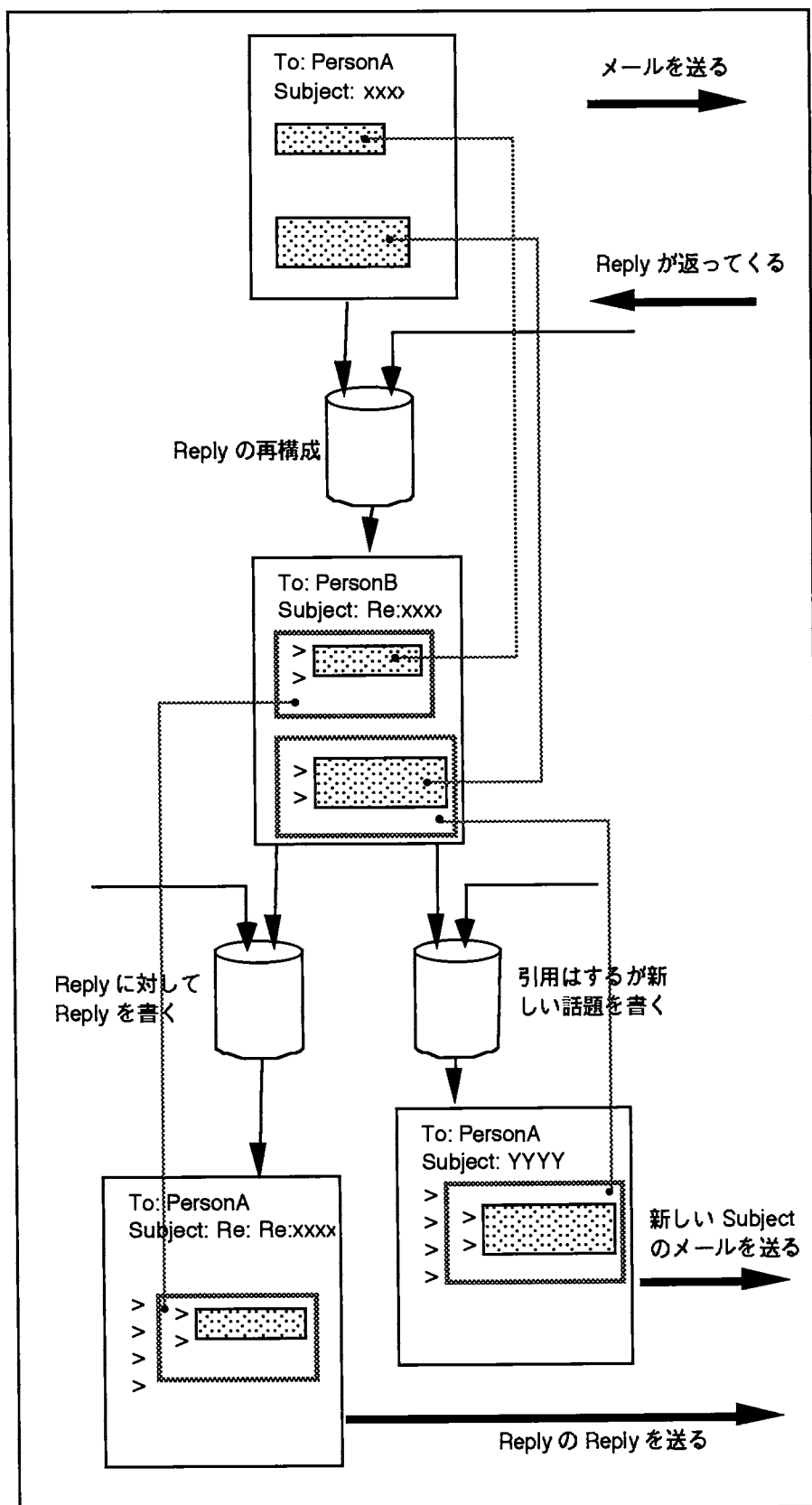
実装する上での課題としては、まずregion の構造クラスについて、どのようなものが必要かという問題がある。ストリーム、ベクタ、整数といった単純な構造はとりあえず実装できるが、さらに複雑な構造については、構造のクラス階層がどうなるか、定義や実装の方法はどうするかあまりわかっていない。

謝辞

本稿で述べた Region 機構に非常にたくさんのアイデアを提供してくれた今井 克暢君に感謝します。

参考文献

- [1] Abelson, Harold, Gerald Jay Sussman, and Julie Sussman. 1985. Structure and Implementation of Computer Programs. MIT Press. (邦訳は、マグロウヒルより、元吉文男訳, 1989)
- [2] Minohara, Tatsuo, and Mario Tokoro. MyAO: A Model for Expressing Persistent Objects. WOOC '90 Working Paper
- [3] Steele, Guy L., Jr. 1990. Common Lisp The Language Second Edition. Digital Press. (邦訳は、共立出版より、井田昌之翻訳監修, 1991)



図? メールに Region 機構を適用した例